Implementation Plan

Vibe Code Security Scanner

Version: 1.0
Author: Jude Perera
Project: Vibe Code Security Scanner
Date: August 2026

1. Project Status

The Vibe Code Security Scanner is feature complete. All planned functionality has been implemented, tested, and committed to GitHub. This document records what was built, in what order, and what remains to be done.

Component	Status	Notes
Database layer	Complete	database.py — scans and findings tables, full CRUD
Python security checks	Complete	14 checks in python_checks.py
Frontend security checks	Complete	2 checks in frontend_checks.py
Scan coordinator	Complete	analyser.py — walks directory, routes by file type
FastAPI backend	Complete	4 routes: GET /, POST /scan, GET /scans, GET /scans/{id}
Web dashboard	Complete	static/index.html — form, results table, scan history
Unit tests	Complete	6 tests, all passing
CI/CD pipeline	Complete	5 jobs, all green
README	Complete	Strix-inspired format with badges and full documentation
AWS EC2 deployment	Pending	Next phase

---

2. Build Order — What Was Built and Why

Phase 1 — Foundation

Step 1: database.py

Built first because everything else depends on it. Defined the two-table schema (scans, findings), connection handling, and all CRUD functions before writing any scanning logic. This ensured the data model was correct before building on top of it.

Step 2: app/checks/python_checks.py — Initial 6 checks

Started with the six most common vibe coding failures: hardcoded secrets, SQL injection, missing auth, debug mode, missing input validation, sensitive data in responses. Each check written as an independent function taking source code and file path, returning a list of findings. Independence was deliberate — allows checks to be tested, added, or removed without touching other checks.

Step 3: app/main.py — FastAPI backend

Built the four API routes once the database and initial checks existed. POST /scan handles the full lifecycle: validate input, clone repo, run analyser, save to database, return results, clean up. Error handling wraps the entire process with a finally block guaranteeing temp directory cleanup.

Step 4: static/index.html — Web dashboard

Built the frontend as a single HTML file with embedded CSS and JavaScript. Fetch API used for all API calls. Dashboard auto-loads scan history on page load. Results render dynamically from JSON responses.

Phase 2 — Security Check Expansion

Steps 5-14: Additional security checks added incrementally

Each new check added as a separate commit with a descriptive message. This produced a meaningful git history showing progressive feature development — a deliberate portfolio decision. Checks added in this phase: missing security headers, environment variables in source, stack trace exposure, missing rate limiting, insecure token storage, exposed admin endpoints, missing file upload validation, API key on frontend, outdated dependencies, IDOR risk, missing 2FA, HTTPS enforcement, server side validation, row level security.

Phase 3 — Refactoring

Project structure refactored into app/ package

Original flat structure (analyser.py, database.py, main.py at root) refactored into a proper Python package structure. Checks split into python_checks.py and frontend_checks.py. analyser.py reduced to a thin coordinator. This demonstrates software engineering discipline — knowing when to refactor, not just when to ship.

Phase 4 — Quality and Documentation

Unit tests

6 unit tests written covering one test per core check category. Tests use temporary files to avoid filesystem coupling. All 6 pass. Tests added to CI/CD pipeline so they run on every push.

CI/CD pipeline

5-job GitHub Actions pipeline: Bandit, Semgrep, Safety dependency audit, Gitleaks secret scanning, pytest unit tests. All jobs run in parallel. All 5 green.

README

Professional README inspired by Strix's format — centred header with badges, problem statement, feature list, quick start, project structure, CI/CD integration example, contributing section, ethical use warning.

---

3. Remaining Work

3.1 AWS EC2 Deployment

Objective

Deploy the application to an AWS EC2 instance so it is accessible via a public URL. This closes the 'system design + deployment' gap in the portfolio.

Steps

- Launch a t2.micro EC2 instance (free tier) in eu-west-2
- Configure security group: allow inbound HTTP (80), HTTPS (443), SSH (22 from your IP only)
- SSH into the instance and install Python 3.11, git, pip
- Clone the repo onto the instance
- Install dependencies: pip install -r requirements.txt
- Run uvicorn with nohup so it persists after SSH disconnect
- Configure nginx as a reverse proxy on port 80 pointing to uvicorn on port 8000
- Record the public IP and add to README as a live demo link

Estimated time

2-3 hours including EC2 setup, nginx configuration, and testing.

3.2 Screenshot

Take a screenshot of the live dashboard showing real scan findings and save as static/screenshot.png. The README references this image but it does not yet exist.

3.3 Medium Article

Write a Medium article explaining the project: what vibe coding is, why it introduces security failures, and how the scanner detects them. Include the SQL injection comparison to explain static analysis in plain English. Link to the GitHub repo.

---

4. Timeline

Phase	Target	Status
Phase 1: Foundation	June 2026	Complete
Phase 2: Check expansion	July 2026	Complete
Phase 3: Refactoring	July 2026	Complete
Phase 4: Quality + docs	August 2026	Complete
Phase 5: Deployment	August 2026	In Progress
Phase 6: Article + LinkedIn	August 2026	Pending
Job applications	August-September 2026	Pending

---

5. Portfolio Positioning

This project demonstrates the following skills directly relevant to Security Engineer graduate roles:

- Python development — 1,000+ lines across multiple modules
- Static analysis — AST-based code parsing, not just regex
- System design — multi-tier web application with database and API
- Web development — FastAPI backend, REST API design, HTML/JS frontend
- DevSecOps — CI/CD pipeline with 5 automated security jobs
- Testing — unit tests with pytest, all passing
- Cloud — AWS EC2 deployment (pending)
- Documentation — PRD, Design Doc, TRD, Implementation Plan

It directly addresses a real, current problem (vibe coding security gaps) that no widely-available open-source tool currently solves. This positions it as original work rather than a tutorial follow-along.